

AI 데이터 분석 · 개발 도구

# Git & GitHub 실전

VS Code로 시작해서 협업까지 — 기록을 남기는 개발자의 기본기

---

.git · 네 개의 공간 · add-commit-push · 소스 제어 창 · 실무 시나리오

---

작업공간 → (add) → Staging → (commit) → 로컬 저장소 → (push) → 원격 저장소  
강의 교안 00\_Git 기반 정리 · GitHub 활동 기록은 곧 개발자 포트폴리오

2026-07 AI 데이터 분석 교육 · Git 정리  
학습자 Minho · 국립금오공과대학교 컴퓨터공학부

## 이 책을 읽는 법

이 책은 강의 교안 00\_Git을 바탕으로, 혼자서도 VS Code에서 Git을 켜서 쓰고 GitHub로 협업하는 흐름까지 한 번에 익히도록 정리한 것입니다. 파란 세로줄은 개념·설명, 회색 세로줄 + **TERMINAL** 은 터미널(Git Bash)에 치는 명령입니다. 뱃지로 "개념 / 명령어 / VS Code 화면 / 협업"을 구분했습니다.

**개념** 원리·구조 **명령어** 터미널 git 명령 **VS Code** 화면·버튼 **협업** 팀·원격 상황

## 목차

|                  | 장 / STEP                       | 무엇을 배우나                                  |
|------------------|--------------------------------|------------------------------------------|
| <b>CHAPTER 0</b> | 왜 Git인가 — 버전 관리와 스냅샷           | 파일명 지옥 · Git=사진 앨범 · 폴더복사 vs Git         |
| <b>STEP 1</b>    | 개발 환경 설정 — 설치부터 VS Code까지      | GitHub 가입 · 설치 · config · git bash · 확장  |
| <b>STEP 2</b>    | .git과 네 개의 공간                  | 작업공간·스테이징·로컬·원격 + 파일의 여정                 |
| <b>STEP 3</b>    | 기본 워크플로우 — add · commit · push | 소스 제어 창 버튼 + 좋은 커밋 메시지                   |
| <b>STEP 4</b>    | 로컬 ↔ 원격, 그리고 작업 흐름             | remote:push-pull-clone + 시퀀스 다이어그램       |
| <b>STEP 5</b>    | .gitignore — 올리면 안 되는 것 거르기    | .env·node_modules · 실전 예시 · 보안           |
| <b>STEP 6</b>    | 실무 시나리오 & 협업                   | status-restore-reset-revert-stash·브랜치·충돌 |

### 실무 팁

처음이라면 CHAPTER 0 → STEP 2(개념) → STEP 3(실습) 순서로 읽고, STEP 4~6은 필요할 때 찾아 보세요. 명령어는 외우려 하지 말고, STEP 6의 "상황 → 명령" 표를 옆에 두고 그때그때 찾아 쓰면 자연스럽게 익습니다.

## CHAPTER 0

# 왜 Git인가 — 버전 관리와 스냅샷

명령어를 외우기 전에, 무엇을 해결하려는 도구인지부터

Git을 배우기 전에 “이게 도대체 무슨 문제를 푸는 도구인가”를 먼저 잡아야 명령어가 머리에 붙습니다. 한 문장으로: Git은 파일의 변경 이력을 스냅샷으로 기록해, 언제·누가·무엇을·왜 바꿨는지 추적하고 되돌릴 수 있게 하는 버전 관리 시스템입니다.

**개념** 버전 관리란? – 파일명 지옥에서 탈출 변경 이력을 기록·관리

**정의** 파일·코드의 변경 이력을 기록하고 관리하는 시스템. 이력이 남아 있으니 이전 버전으로 되돌리거나 여러 버전을 비교·병합할 수 있습니다.

```
project.html
project_backup.html
project_final.html
project_진짜최종.html
project_진짜최종_v2.html ← 어느 게 최신?!
```

**왜 쓰나** 파일명에 backup-final-진짜최종을 붙이다 보면 폭발합니다. ① 어느 게 최신인지, ② 무엇이 왜 바뀌었는지, ③ 여러 명이 고치면 누구 것이 맞는지 — 이 세 가지를 사람 손으로는 관리할 수 없습니다. 그걸 자동으로 해주는 것이 버전 관리입니다.

**개념** Git = 스냅샷 (사진 앨범 비유) 저장할 때마다 그 시점 전체를 기록

**정의** Git은 커밋할 때마다 그 순간의 프로젝트 전체 상태를 “스냅샷(사진)”으로 남깁니다.

**왜 쓰나** 스마트폰 사진 앨범을 떠올리세요. 사진을 찍을 때마다 그 순간이 통째로 남고, 언제든지 그 시점으로 돌아갈 수 있습니다. Git의 커밋이 바로 그 “사진 한 장”이고, 각 사진에는 언제·누가·무엇을 바꿨는지가 함께 기록됩니다.

### 개념

폴더 복사 백업과의 차이: ①변경 내용을 줄 단위로 자동 추적, ②변경 이유를 커밋 메시지로 기록, ③전체가 아니라 “변경분만” 저장해 가벼움, ④명령어 하나로 과거로 즉시 복원. 폴더 복사로는 이 넷을 다 못합니다.

**협업** 왜 지금 배우나 – 포트폴리오가 된다 GitHub 활동 = 실력 증명

**정의** Git과 연동한 GitHub의 커밋 기록·개인 저장소는 개발자의 실력을 보여주는 가장 좋은 자료입니다.

**왜 쓰나** 꾸준한 커밋 잔디와 프로젝트 저장소가 곧 이력서가 됩니다. 백엔드·임베디드·DX/AX 어느 방향이든 GitHub는 취업에서 강력한 무기이고, 팀 프로젝트에서는 없으면 협업 자체가 안 됩니다. 오늘 만든 코드를 저

장소로 만들어 커밋해두는 것부터가 그 시작입니다.

#### 실무 팁

이번 시간에 함께 쓰는 도구는 셋입니다 — Git(버전 관리 엔진), GitHub(원격 저장소 플랫폼), VS Code(편집기). 세 개가 손발을 맞춰 돌아갑니다.

## 폴더 복사 백업 vs Git — 무엇이 다른가

“그냥 폴더를 날짜별로 복사해두면 안 되나?”라는 의문이 들 수 있습니다. 실제로 많이들 그렇게 하지만, 아래 네 가지에서 확실히 갈립니다.

| 비교 항목    | 폴더 복사 방식          | Git              |
|----------|-------------------|------------------|
| 변경 내용 파악 | 파일을 직접 열어 눈으로 비교  | 줄 단위로 자동 추적·기록   |
| 변경 이유 기록 | 알 수 없음 (파일명에만 의존) | 커밋 메시지로 “왜”까지 기록 |
| 저장 공간    | 매번 전체 복사 → 용량 폭증  | 바뀐 부분(변경분)만 저장   |
| 과거로 되돌리기 | 폴더 찾아 수동 복원       | 명령어 하나로 즉시 복원    |

#### 개념

정리하면, 폴더 복사는 “결과물만” 쌓지만 Git은 “변화의 이야기(누가·언제·왜·무엇을)”를 쌓습니다. 그래서 예측 정비 코드처럼 오래 관리해야 하는 프로젝트일수록 Git의 가치가 커집니다. 되돌릴 수 있다는 안정감 덕분에 “일단 해보고 아니면 되돌리자”라는 과감한 실험도 가능해집니다.

## 세 도구의 역할 나누기

| 도구      | 무엇을 하나                         | 어디에 있나    |
|---------|--------------------------------|-----------|
| Git     | 버전 관리 엔진 — 커밋·브랜치·히스토리 관리      | 내 컴퓨터(로컬) |
| GitHub  | 원격 저장소 플랫폼 — 백업·공유·협업·PR       | 인터넷 서버    |
| VS Code | 편집기 — 코드 작성 + 소스 제어 창으로 Git 조작 | 내 컴퓨터(화면) |

#### 개념

셋의 관계 한 줄: VS Code에서 코드를 쓰고 → Git이 그 변화를 기록하고 → GitHub가 그 기록을 인터넷에 보관·공유합니다. 오늘은 이 셋을 손발 맞춰 쓰는 법을 배웁니다.

## STEP 1

# 개발 환경 설정 — 설치부터 VS Code까지

한 번만 해두면 계속 쓰는 준비 작업

Git은 “한 번 설치하고 한 번 설정”해두면 그 뒤로는 명령만 쓰면 됩니다. 순서는 ① GitHub 가입 → ② Git 설치 → ③ 최초 설정(git config) → ④ VS Code에서 Git Bash 열기입니다.

**개념** 준비물 3 가지    GitHub · Git · VS Code

**정의** GitHub 계정(원격 저장소), Git 프로그램(내 컴퓨터의 버전 관리 엔진), VS Code(편집기 + 확장)를 준비합니다.

```
GitHub 가입 : https://github.com
Git 설치(Win) : https://git-scm.com → 설치 시 "Git Bash" 함께 설치됨
Git 설치(Mac) : brew install git (Homebrew)
VS Code 확장 : GitLens · Auto Commit Message
```

**왜 쓰나** GitHub 가입 이메일은 뒤의 git config 이메일과 똑같이 맞추는 게 좋습니다(커밋 주인이 GitHub 계정과 연결됨). Windows에서 Git을 설치하면 “Git Bash”라는 터미널이 함께 깔리는데, 이게 강의·실무 명령어가 그대로 통하는 셸입니다.

**명령어** git config – 최초 1 회 설정    나를 등록

**정의** Git 첫 설치 후 딱 한 번, 커밋에 남길 이름과 이메일을 등록합니다. --global은 “이 컴퓨터 전체에 한 번만”이라는 뜻입니다.

```
TERMINAL (Git Bash)
git config --global user.name "Minho"
git config --global user.email "gh계정@example.com"
git config --global init.defaultBranch main # 기본 브랜치 이름 main
git config --global core.editor "code --wait" # 기본 편집기 VS Code (선택)
git config --global --list # 설정 확인
```

**왜 쓰나** 앞으로 남길 모든 커밋에 “누가 언제” 했는지가 이 정보로 붙습니다. 한 번만 해두면 다시 할 필요 없습니다.

### 실무 팁

 Windows 추가 권장: git config --global core.autocrlf true — 윈도우/맥 사이 줄바꿈 (CRLF/LF) 차이를 자동 변환해 “파일 전체가 바뀐 것처럼” 보이는 문제를 막아줍니다.

## VS Code VS Code 에서 Git Bash 터미널 열기 내 편집기 안에서 명령 치기

**정의** VS Code 안의 통합 터미널을 Git Bash로 열면, 편집과 git 명령을 같은 화면에서 처리할 수 있습니다.

- ① 터미널 열기 : Ctrl + ` (백틱) 또는 메뉴 Terminal → New Terminal
- ② 셸 고르기 : 터미널 우측 위 [ + ] 옆 드롭다운(v) 클릭
- ③ Select Default Profile → "Git Bash" 선택
- ④ 새 터미널을 열면 이제 Git Bash로 열림 (프롬프트에 \$ 표시)

**왜 쓰나** 기본 CMD/PowerShell 대신 Git Bash를 쓰면 ls, rm, pwd 같은 유닉스 명령이 그대로 통해, 강의·실무에서 보는 명령어를 똑같이 따라 칠 수 있습니다. Git Bash가 목록에 없으면 Git for Windows 설치가 안 된 것입니다.

### 개념

확인 습관: 터미널을 연 뒤 git --version으로 설치를, pwd로 지금 위치한 폴더를 먼저 확인하세요. 엉뚱한 폴더에서 git init 하는 실수를 줄여줍니다.

## VS Code 확장 – GitLens · Auto Commit Message 있으면 편한 도구

**정의** 필수는 아니지만, 처음 배울 때 Git을 "눈으로" 이해하게 도와주는 확장입니다.

GitLens : 코드 각 줄을 누가·언제 바꿨는지 인라인 표시 + 히스토리 시각화  
Auto Commit Message : 변경 파일을 분석해 커밋 메시지 초안을 자동 생성

**왜 쓰나** GitLens는 "이 줄을 언제 왜 고쳤지?"를 편집기 안에서 바로 보여줘, 남이 쓴 협업 코드를 읽을 때 특히 강력합니다. 확장은 VS Code 왼쪽 Extensions(네모 4개) 아이콘에서 이름으로 검색해 설치합니다.

## 설치·설정 순서 한눈에

| 순서 | 할 일                         | 한 번만?   |
|----|-----------------------------|---------|
| 1  | GitHub 가입 (github.com)      | 계정당 1회  |
| 2  | Git 설치 (git-scm.com / brew) | 컴퓨터당 1회 |
| 3  | git config로 이름·이메일 등록       | 컴퓨터당 1회 |
| 4  | VS Code 기본 터미널을 Git Bash로   | 1회      |
| 5  | (선택) GitLens 등 확장 설치        | 1회      |

#### 실무 팁

여기까지는 “한 번만” 해두면 되는 준비 작업입니다. 이 다섯 개를 마치면, 그 뒤로는 STEP 3의 **add·commit·push** 리듬만 반복하면 됩니다. 새 컴퓨터를 쓰게 되면 2~4번만 다시 하면 되고요.

#### 주의

##### 설치·인증에서 자주 겪는 문제

- 터미널 드롭다운에 Git Bash가 안 보임 → Git for Windows 설치가 안 됐거나 VS Code 재시작이 필요.
- push할 때 아이디/비밀번호를 물어봄 → 비밀번호가 아니라 PAT(토큰) 또는 `gh auth login`.
- 커밋 작성자가 엉뚱하게 찍힘 → `git config --global user.name/email`을 GitHub 계정과 맞췄는지 확인.

## STEP 2

# .git과 네 개의 공간

이 개념만 잡으면 모든 명령이 “어디서 어디로”인지 보인다

Git 명령이 헛갈리는 이유는 파일이 “보이지 않는 여러 공간”을 오가기 때문입니다. 이 공간 지도를 먼저 그리면, 모든 명령이 “어느 공간에서 어느 공간으로 보내는가”로 정리됩니다.

**명령어** `git init` 과 `.git` 폴더 폴더를 저장소로 만들기

**정의** `git init`을 하면 그 폴더에 숨김 폴더 `.git/`이 생기고, 그 순간부터 폴더가 “Git이 관리하는 저장소”가 됩니다.

### TERMINAL (Git Bash)

```
git init          # 현재 폴더를 로컬 저장소로 만들  
ls -a             # 숨김 폴더 .git 확인
```

**왜 쓰나** `.git/` 폴더가 곧 “로컬 저장소(local repository) 그 자체”입니다. 지금까지의 모든 커밋(스냅샷)·브랜치·설정·히스토리가 전부 이 안에 들어 있습니다. 폴더에 `.git`이 있으면 Git이 관리하는 폴더, 없으면 그냥 평범한 폴더입니다.

### 주의

`.git` 폴더는 절대 수동으로 열어서 수정하거나 삭제하지 마세요. 삭제하면 지금까지 쌓은 모든 이력이 복구 불가능하게 사라지고, 그냥 일반 폴더로 돌아갑니다.

## 네 개의 공간 — 파일이 오가는 지도

| 공간                                            | 정체                                       | 여기로 보내는 명령              |
|-----------------------------------------------|------------------------------------------|-------------------------|
| ① 작업공간 Working Directory                      | 내가 실제로 파일을 만들고 고치는 폴더 (VS Code에 보이는 파일들) | (편집)                    |
| ② 스테이징 Staging Area                           | 이번 커밋에 넣을 것만 골라 담은 “대기 무대”               | <code>git add</code>    |
| ③ 로컬 저장소 Local Repository = <code>.git</code> | 내 컴퓨터에 커밋(스냅샷)이 쌓이는 곳 (인터넷 없이도 동작)       | <code>git commit</code> |
| ④ 원격 저장소 Remote Repository = GitHub           | 인터넷 서버에 있는 저장소 (백업·공유·협업)                | <code>git push</code>   |



## 개념

### 용어 3형제 확실히 구분하기

- **local directory (작업 폴더)** = 내 눈에 보이는 실제 파일들. 내가 편집하는 그 폴더.
- **local repository (로컬 저장소)** = 그 폴더 안 `.git`에 쌓인 커밋 기록. 내 컴퓨터 안, 인터넷 필요 없음.
- **remote repository (원격 저장소)** = GitHub 서버의 저장소. 백업 + 다른 사람과 공유.

## 주의

핵심: **로컬과 원격은 서로 독립**입니다. 로컬에서 커밋해도 자동으로 GitHub에 올라가지 않고(→ `git push` 필요), GitHub에서 바뀐 것도 자동으로 내려오지 않습니다(→ `git pull` 필요). 둘을 잇는 다리가 push와 pull입니다.

## 파일 하나가 네 공간을 지나가는 여정

개념을 실제 명령으로 이어봅시다. 새 파일 하나가 작업공간 → 스테이징 → 로컬 저장소로 옮겨가는 동안, `git status`가 매 단계 무엇을 알려주는지 따라가면 네 공간이 손에 잡힙니다.

### TERMINAL (Git Bash)

```
$ echo "hello" > note.txt          # ① 작업공간에 새 파일 생성
$ git status
Untracked files:  note.txt        ← Git이 아직 모르는 새 파일

$ git add note.txt                 # ② 스테이징(무대)으로 올림
$ git status
Changes to be committed: note.txt ← 커밋 대기(초록)

$ git commit -m "docs: note 추가" # ③ 로컬 저장소에 스냅샷 저장
$ git status
nothing to commit, working tree clean ← 다 기록됨!
```

## 개념

같은 파일이 **Untracked → Staged → Committed**로 "상태"를 바꿔 갑니다. `git status`는 지금 파일이 어느 공간에 있는지 알려주는 나침반이라, 헛갈릴 때마다 치면 됩니다.

**개념** 왜 스테이징(무대)이 따로 있나 "고르는 단계"의 가치

**정의** 작업공간에서 바로 커밋하지 않고 add로 한 번 "고르는" 단계를 두는 데는 이유가 있습니다.

**왜 쓰나** 10개 파일을 고쳤어도 이번 커밋엔 서로 관련된 3개만 담고 싶을 때가 많습니다. 스테이징이 있어서 "이번 사진에 넣을 것"만 골라 담고, 나머지는 다음 커밋으로 미룰 수 있습니다. 커밋을 의미 단위로 깔끔하게

쪼개는 힘이 바로 여기서 나옵니다.

## 파일의 네 가지 상태

`git status`가 파일을 분류하는 네 가지 상태입니다. 소스 제어 창 의 U·M·A 글자와 그대로 연결됩니다.

| 상태        | 뜻                    | 다음 행동                   |
|-----------|----------------------|-------------------------|
| Untracked | Git이 아직 모르는 새 파일     | <code>git add</code>    |
| Modified  | 추적 중인데 수정된 파일        | <code>git add</code>    |
| Staged    | add 후 — 커밋 대기        | <code>git commit</code> |
| Committed | commit 후 — 스냅샷 저장 완료 | (완료)                    |

### 개념

이 네 상태가 STEP 3에서 배울 `add·commit`의 "대상"입니다. `add`는 Untracked/Modified를 Staged로, `commit`은 Staged를 Committed로 옮깁니다.

### STEP 3

## 기본 워크플로우 — add · commit · push

매일 쓰는 세 동작, 터미널과 소스 제어 창 두 방법으로

Git에서 매일 반복하는 리듬은 딱 셋입니다 — 골라 담고(add), 사진 찍고(commit), 올린다(push). 먼저 터미널 명령으로 익히고, 그다음 VS Code 소스 제어 창의 버튼이 각각 어떤 명령인지 하나씩 연결합니다.

**명령어** `git add · commit · push` 골라 담고 → 찍고 → 올린다

**정의** add는 변경 파일을 스테이징에 올리고, commit은 그것을 스냅샷으로 확정하며, push는 원격(GitHub)에 올립니다.

#### TERMINAL (Git Bash)

```
git status          # (수시로) 지금 상태 확인
git add .           # 변경분 전부 스테이징 (특정 파일: git add 파일명)
git commit -m "feat: 로그인 화면 추가"  # 스냅샷 + 메시지
git push            # 원격에 올리기
git log --oneline   # 커밋 이력 한 줄로 보기 (q로 종료)
```

```
▶ [main 3f9a1c2] feat: 로그인 화면 추가
▶ 1 file changed, 12 insertions(+)
```

**왜 쓰나** add와 commit이 나뉜 이유: 10개를 고쳤어도 이번 커밋엔 관련된 3개만 담고 싶을 때가 있습니다. add로 "고르고" commit으로 "확정"합니다. 커밋 전 `git status`로 무엇이 담겼는지 확인하는 게 습관입니다.

#### 실무 팁

한 방에 하기: `git commit -am "메시지"`는 add + commit을 한 번에 (단, 이미 추적 중인 파일만).

## VS Code 소스 제어 창 — 버튼의 정체

왼쪽 사이드바의 가지(branch) 모양 아이콘, 또는 `Ctrl + Shift + G`를 누르면 소스 제어(Source Control) 창이 열립니다. 여기 버튼들은 사실 전부 git 명령의 다른 얼굴입니다.

| 소스 제어 창에서                  | 하는 일              | 같은 터미널 명령                            |
|----------------------------|-------------------|--------------------------------------|
| 상단 메시지 입력칸 + <b>Commit</b> | 메시지 쓰고 커밋         | <code>git commit -m "..."</code>     |
| 파일 옆  (Stage Changes)      | 그 파일을 스테이징에 올림    | <code>git add 파일</code>              |
| 파일 옆  (Unstage)            | 스테이징에서 내림         | <code>git restore --staged 파일</code> |
| 파일 옆  (Discard Changes)    | 변경 취소(마지막 커밋 상태로) | <code>git restore 파일</code>          |

| 소스 제어 창에서                          | 하는 일                | 같은 터미널 명령                        |
|------------------------------------|---------------------|----------------------------------|
| ... 메뉴 → <b>Sync / Push / Pull</b> | 원격과 주고받기            | <code>git push / git pull</code> |
| 파일 이름 클릭                           | 바뀐 줄(diff)을 좌·우로 비교 | <code>git diff</code>            |

### 개념

파일 앞 상태 글자도 알아두면 좋습니다: **U**(Untracked·새 파일) · **M**(Modified·수정됨) · **A**(Added·스태이징됨) · **D**(Deleted) · **C**(Conflict·충돌). 터미널의 ``git status``가 알려주는 것과 똑같은 정보를, 색과 글자로 보여주는 것뿐입니다.

### 실무 팁

터미널 방식과 소스 제어 창 방식은 **같은 .git을 건드리는 동일한 작업**입니다. 커밋은 창에서 클릭으로, 로그·되돌리기는 터미널로 — 편한 대로 섞어 써도 됩니다. 처음엔 창으로 감을 잡고, 익숙해지면 터미널이 더 빠릅니다.

## 좋은 커밋 메시지

커밋 메시지는 “미래의 나와 팀원”이 읽는 기록입니다. 한 달 뒤에도 무엇을·왜 했는지 알 수 있어야 합니다.

| X 나쁜 예 | 왜 별로인가          | ✓ 좋은 예                |
|--------|-----------------|-----------------------|
| "수정"   | 무엇을 고쳤는지 알 수 없음 | "fix: 로그인 버튼 오류 수정"   |
| "o"    | 아무 의미 없음        | "feat: 회원가입 API 추가"   |
| "작업함"  | 무슨 작업인지 불명      | "docs: README 사용법 보강" |

### 실무 팁

관례로 “타입: 내용” 형식이 널리 쓰입니다 — `feat`(기능), `fix`(버그), `docs`(문서), `refactor`(리팩터). 그리고 커밋은 “의미 단위”로 자주 남기세요. 하루 끝에 몰아서 하나로 커밋하기보다, 기능 하나 끝날 때마다 잘게 남기는 편이 되돌리기도 읽기도 쉽습니다.

## STEP 4

# 로컬 ↔ 원격, 그리고 작업 흐름

push-pull-clone과 교안 21p 시퀀스 다이어그램 완전 해부

로컬 저장소(.git)와 원격 저장소(GitHub)는 독립적이라, 명시적으로 “올리고(push) 내려받아야(pull)” 합니다. 이 왕복을 그림 하나로 요약한 것이 강의 교안 21페이지의 시퀀스 다이어그램입니다. 먼저 그림을 보고, 화살표를 하나씩 뜯어봅니다.

그림. Git 작업 흐름 (00\_Git 교안 p.21 재구성)



## 시퀀스 다이어그램 한 줄씩 읽기

위 그림에서 위쪽 네 개의 원기둥은 파일이 머무는 네 “공간”이고, 아래로 내려오는 세로선은 그 공간의 시간 흐름입니다. 가로 화살표가 “어떤 명령이 어느 공간에서 어느 공간으로 파일을 옮기는가”를 보여줍니다. 원→오른쪽이 “올리는” 방향, 맨 아래 긴 화살표만 오른→왼쪽으로 “내려받는” 방향입니다.

**명령어** ① `git add` – 작업공간 ▶ 스테이징 노란 화살표

**정의** 작업공간에서 바뀐 파일 중 “이번에 기록할 것”만 골라 스테이징(무대)에 올립니다.

**왜 쓰나** 그림에서 가장 왼쪽 화살표입니다. 아직 사진을 찍은 건 아니고, 사진에 담을 대상을 무대에 세워두는 단계예요. 여러 번 add해서 무대를 구성할 수 있습니다.

**명령어** ② `git commit` – 스테이징 ▶ 로컬 저장소 초록 화살표

**정의** 무대에 올라온 것들을 하나의 스냅샷(커밋)으로 확정해 로컬 저장소(.git)에 저장합니다.

**왜 쓰나** 이 순간 “사진 한 장”이 찍혀 히스토리에 남습니다. 언제·누가·무엇을(메시지)이 함께 기록되죠. 여기까지는 전부 내 컴퓨터 안에서만 일어난 일 — 아직 GitHub는 이 커밋을 모릅니다.

**명령어** ③ `git push` – 로컬 저장소 ▶ 원격 저장소 파란 화살표

**정의** 로컬 저장소에 쌓인 커밋들을 원격 저장소(GitHub)로 올립니다.

### TERMINAL (Git Bash)

```
git remote add origin https://github.com/아이디/저장소.git # 원격 주소 연결(1회)
git push -u origin main # 첫 push (-u: 기본 원격 설정)
git push # 이후에는 이것만
```

**왜 쓰나** 이제야 내 커밋이 인터넷에 올라가 백업되고, 팀원도 볼 수 있게 됩니다. 처음 한 번은 원격 주소를 연결(remote add)하고 -u로 기본 경로를 정해두면, 그 뒤론 `git push`만으로 올라갑니다.

#### 실무 팁

인증은 비밀번호가 아니라 PAT(Personal Access Token) 또는 `gh auth login`으로 합니다. push 시 Username=GitHub 아이디, Password 자리에 발급받은 토큰을 붙여넣습니다.

**명령어** ④ `git pull` - 원격 저장소 ▶ 작업공간 (역방향) 맨 아래 긴 빨간 화살표

**정의** 원격의 최신 변경을 가져와 로컬 저장소를 거쳐 내 작업공간까지 한 번에 반영합니다.

#### TERMINAL (Git Bash)

```
git pull                # 원격 최신 받아 현재 브랜치에 반영
# (git pull = git fetch + git merge 를 한 번에)
```

**왜 쓰나** 그림에서 유일하게 오른쪽→왼쪽으로, 그것도 원격에서 작업공간까지 길게 가로지르는 이유가 여기 있습니다. 원격의 새 커밋을 로컬 저장소로 내려받고(fetch), 그것을 내 작업공간 파일에 합치는(merge) 두 일을 pull 하나가 한꺼번에 처리하기 때문입니다.

#### 개념

방향으로 외우기: add-commit-push는 “오른쪽으로 밀어 올리기”, pull은 “왼쪽으로 당겨 내려받기”. 그래서 협업의 하루는 보통 `git pull`(내려받아 시작) → 작업 → `add`→`commit`→`push`(올려서 마무리) 순서입니다.

**명령어** (참고) `git clone` - 원격을 통째로 복제 처음 받아올 때

**정의** 원격 저장소 전체를 내 컴퓨터로 복사합니다. origin 연결도 자동으로 됩니다.

#### TERMINAL (Git Bash)

```
git clone https://github.com/아이디/저장소.git
```

**왜 쓰나** 새 컴퓨터에서 기존 프로젝트를 처음 가져올 땐 `init`이 아니라 `clone`입니다. 다이어그램으로 치면 “원격 → 로컬을 통째로 새로 만드는” 특별한 첫 내려받기예요.

## STEP 5

# .gitignore — 올리면 안 되는 것 거르기

비밀번호·쓰레기 파일이 GitHub에 올라가는 사고 막기

모든 파일을 다 올리면 안 됩니다. 비밀번호가 든 설정 파일, 수천 개짜리 라이브러리 폴더, OS가 만든 부산물 등은 추적에서 빼야 합니다. 그 “제외 목록”이 `.gitignore` 파일입니다.

**개념** `.gitignore` 기본 추적 제외 목록

**정의** 저장소 루트(최상위 폴더)에 두는 텍스트 파일. 여기 적은 파일·폴더를 Git이 “아예 없는 것처럼” 무시합니다.

```
.gitignore
# 폴더 통째로 제외 (뒤에 / )
node_modules/
Lecture/

# 특정 확장자
*.log
*.env

# OS·편집기 부산물
.DS_Store      # macOS
.vscode/
```

**왜 쓰나** `.env`(API 키·비밀번호)가 GitHub에 올라가면 전 세계에 공개됩니다. `node_modules/`는 수천 개 파일이라 저장소를 무겁게 만들고요. 이런 것들을 목록에 적어두면 `add-commit` 대상에서 자동으로 빠집니다.

**개념** 무엇을 빼야 하나 – 세 부류 민감 정보·자동 생성물·대용량

**정의** “다시 만들 수 있는 것”과 “올라가면 위험한 것”이 기준입니다.

|         |                                                                      |
|---------|----------------------------------------------------------------------|
| 민감한 설정  | <code>.env</code> (API 키·비밀번호·토큰) → 유출되면 사고                          |
| 시스템 부산물 | <code>.DS_Store</code> (맥) · <code>Thumbs.db</code> (윈도우)            |
| 라이브러리   | <code>node_modules/</code> · <code>.venv/</code> (다시 설치하면 되는 것)      |
| 빌드 결과물  | <code>dist/</code> · <code>build/</code> · <code>__pycache__/</code> |

**왜 쓰나** 소스 코드와 설정 “예시”(예: `env.example`)는 올리고, 실제 비밀 값(`.env`)은 빼니다. 라이브러리 폴더는 남이 clone한 뒤 `pip install`·`npm install`로 다시 받으면 되므로 굳이 올리지 않습니다.

**주의** 이미 커밋된 파일은 `.gitignore` 로 안 빠진다 가장 흔한 함정

**정의** `.gitignore`는 “아직 추적 안 하는 파일”에만 적용됩니다. 이미 한 번 커밋된 파일은 목록에 적어도 계속

추적됩니다.

#### TERMINAL (Git Bash)

```
# 이미 올라간 파일을 추적에서 빼려면:
git rm -r --cached .env          # 추적만 해제 (로컬 파일은 그대로 유지)
echo ".env" >> .gitignore        # 목록에 추가
git commit -m "chore: .env 추적 제외"
```

**왜 쓰나** `--cached`가 핵심입니다. 이게 있으면 "Git의 추적에서만" 빼고 내 디스크의 실제 파일은 지우지 않습니다. 실수로 커밋해버린 비밀 파일을 뒤늦게 빼는 표준 절차예요.

#### 주의

이미 push까지 된 비밀번호·토큰은 추적을 빼도 "과거 커밋 히스토리"에는 남아 있습니다. 유출된 토큰은 반드시 GitHub에서 재발급(무효화)하세요. 추적 해제만으로는 안전해지지 않습니다.

## 실전 .gitignore 예시 (파이썬·웹)

처음엔 아래를 통째로 복사해 두고, 프로젝트에 맞게 줄여 쓰면 됩니다. 저장소 루트에 `.gitignore` 파일로 저장합니다.

```
.gitignore
# — 비밀·설정 —
.env
*.key
# — 파이썬 —
__pycache__/
*.pyc
.venv/
# — 노드/웹 —
node_modules/
dist/
# — OS·편집기 —
.DS_Store
Thumbs.db
.vscode/
```

#### 보안

가장 중요한 한 줄은 `.env` 입니다. API 키·DB 비밀번호가 여기 들어가는데, 공개 저장소에 올라가면 봇이 몇 분 만에 훑어갑니다. 새 프로젝트를 만들면 **코드 한 줄 쓰기 전에 .gitignore부터** 만드는 습관을 들이세요.



## STEP 6

# 실무 시나리오 & 협업

“이런 일이 생기면 이 명령” — 협업에서 가장 중요한 부분

명령어를 따로 외우는 것보다, “이런 상황엔 이걸 친다”로 익히는 게 실전에 강합니다. 아래 표가 이 책에서 가장 자주 펴 볼 페이지가 될 거예요.

| 이런 상황이 생기면             | 이렇게 한다                                                    |
|------------------------|-----------------------------------------------------------|
| 오늘 작업 시작 (최신부터 받고 시작)  | <code>git pull</code>                                     |
| 뭔가 바뀌었는지 모르겠다          | <code>git status</code> → 자세히는 <code>git diff</code>      |
| 한 것을 기록해두고 싶다          | <code>git add .</code> → <code>git commit -m "..."</code> |
| GitHub에 올리고 싶다         | <code>git push</code>                                     |
| 파일을 잘못 고쳤다 (아직 커밋 전)   | <code>git restore</code> 파일명                              |
| add를 잘못했다 (스테이징만 취소)   | <code>git restore --staged</code> 파일명                     |
| 방금 커밋 메시지 오타           | <code>git commit --amend</code>                           |
| 방금 커밋 자체를 무르기 (변경은 유지) | <code>git reset --soft HEAD~1</code>                      |
| 이미 push한 커밋을 안전하게 되돌리기 | <code>git revert &lt;커밋&gt;</code>                        |
| 급히 브랜치 바꿔야 한다 (커밋 전)   | <code>git stash ...</code> 나중에 <code>git stash pop</code> |
| 새 기능을 따로 작업            | <code>git switch -c feature/이름</code>                     |
| push했더니 거부됨(원격에 새 커밋)  | <code>git pull</code> 먼저 → 합치고 다시 <code>git push</code>   |

**협업** 충돌(conflict) 만났을 때 겁먹지 말 것

**정의** 같은 줄을 나와 남이 다르게 고쳐 합쳐질 때 충돌이 납니다. Git이 그 부분을 표시해주면, 사람이 골라서 정리합니다.

```
<<<<<<< HEAD
내가 쓴 내용
=====
남이 쓴 내용
>>>>>>> main
```

# → 표시 줄들을 지우고 최종본만 남긴 뒤

`git add 파일 && git commit` # 충돌 해결 커밋

**왜 쓰나** <<<<, ==, >>>> 사이에서 "내 것/남의 것/둘 다"를 직접 골라 정리하고, add→commit 하면 끝입니다. VS Code는 이 부분에 "Accept Current / Incoming / Both" 버튼을 띄워줘 클릭으로도 해결됩니다.

**협업** 브랜치 습관 – 많이 만드는 건 OK, 방치가 문제 짧게 살고 자주 병합

**정의** 브랜치는 특정 커밋을 가리키는 포인터(가벼운 텍스트 하나)라, 많이 만들어도 부담이 거의 없습니다. 진짜 비용은 "개수"가 아니라 "수명"입니다.

#### TERMINAL (Git Bash)

```
git switch -c feature/login # 작업 브랜치 생성+이동
# ... 작업 ...
git push -u origin feature/login
# GitHub에서 Pull Request → 리뷰 → merge
git switch main && git pull
git branch -d feature/login # 끝난 브랜치 삭제
```

**왜 쓰나** 오래 방치된 브랜치는 main과 멀어져 나중에 충돌 지옥이 됩니다. 건강한 원칙은 "짧게 살고 → 자주 병합 → 끝나면 삭제", 이름은 feature/, fix/처럼 규칙을 둡니다. 혼자 학습용 레포면 main에서 바로 작업해도 큰 문제 없지만, 지금부터 브랜치 흐름을 연습해두면 팀 협업·취업 대비에 좋습니다.

## 전형적인 하루 흐름

#### TERMINAL — 협업하는 하루

```
git pull # 1. 최신 상태로 맞추기
git switch -c feature/login # 2. 작업 브랜치 생성
# ... 코드 작업 ...
git status # 3. 뭐 바뀌었나 확인
git add . # 4. 스테이징
git commit -m "feat: 로그인 추가" # 5. 커밋
git push -u origin feature/login # 6. 원격에 올리기
# 7. GitHub에서 Pull Request 생성 → 리뷰 → merge
```

#### 실무 팁

막히면 항상 git status 부터. 지금 상태(무엇이 스테이징됐고 무엇이 안 됐는지)와 다음에 뭘 하면 되는지를 Git이 친절하게 알려줍니다. 오늘 만든 코드로 git init → add → commit "study: git 첫 커밋"까지 직접 해보는 것이 진짜 마무리입니다.